

Project Utilities and Libraries

Desktop – Electron + React + TypeScript

Core Setup

- `electron` – desktop shell that creates the tray icon, timer window, and settings UI on Windows, macOS, and Ubuntu
- `electron-builder` – packages the Electron app into OS-specific installers: EXE/NSIS for Windows, DMG/PKG for macOS, AppImage/DEB for Linux
- `electron-updater` – handles automatic in-app updates so employees always run the latest agent version without manual reinstallation
- `electron-store` – persists local config (screenshot interval, idle timeout, last login) to disk across app restarts
- `electron-log` – writes structured log files on the employee machine for debugging agent issues without opening DevTools
- `auto-launch` – registers the Electron app to start silently at OS login so monitoring begins automatically

UI Framework

- `react` – builds the component-based UI for the timer window, settings panel, and onboarding screens
- `react-dom` – renders React components into the Electron BrowserWindow renderer process
- `typescript` – adds static type safety across all Electron main and renderer code
- `vite` – fast build tool and dev server for the Electron renderer (React + TS) with HMR
- `react-router-dom` – handles client-side routing between timer, settings, login, and diagnostics pages inside Electron

UI Components & Styling

- `@mui/material` or `antd` – ready-made accessible UI components (tables, modals, buttons, forms) used across the timer and settings UI
- `tailwindcss` – utility-first CSS for custom layouts and spacing without writing separate CSS files

- `lucide-react` – lightweight icon library used for tray menu icons, status indicators, and button icons
- `@primer/octicons-react` – GitHub's official icon set used to visually distinguish push, PR, review, issue, and branch events in the GitHub activity feed

Charts & Visualization

- `recharts` or `echarts-for-react` – renders productivity graphs, active vs idle time charts, and weekly trend lines on both admin and employee dashboards
- `react-calendar-heatmap` – displays a GitHub-style calendar heatmap for attendance and daily activity intensity

State Management

- `zustand` – lightweight global state store for org context, current user session, active filters, and work-mode toggle status
- `@tanstack/react-query` – manages async API calls with caching, background refetch, loading and error states for all dashboard data

Communication

- `axios` – HTTPS client for all REST API calls to the backend (login, reports, config fetch, GitHub feed)
 - Native `WebSocket` – used for real-time live tracking updates and WebRTC signaling channel between the daemon and admin browser
 - Native `RTCPeerConnection` – browser WebRTC API used by the admin dashboard to receive the live employee screen stream from the TURN relay
 - HTML `<video>` with `srcObject` – renders the incoming WebRTC live screen stream in the admin live view panel (read-only, no interaction)
-

Rust Monitoring Daemon (Agent)

Core Runtime & Async

- `tokio` – async runtime that powers all concurrent tasks: event collection, screenshot capture, upload queue, and IPC server simultaneously
- `serde` – serialization framework used to convert Rust structs to/from JSON for all API payloads and local SQLite storage

- `serde_json` – JSON serializer/deserializer built on `serde`; used for activity event payloads sent to backend

OS Integrations

- `windows` – Win32 API bindings used to get the foreground window, process name, window title, and capture screenshots on Windows
- `core-foundation` + `cocoa` – macOS Cocoa and CoreFoundation bindings used to access `NSWorkspace` and `Accessibility` APIs for active app and window title on macOS
- `x11rb` or `xcb` – X11 protocol bindings used to detect the active window and window title on Linux/X11 desktops
- `screenshots` – cross-platform crate for capturing screen frames on Windows, macOS, and Linux for the periodic screenshot feature

Live Screen Streaming

- `webrtc` (`webrtc-rs`) – pure Rust WebRTC implementation that encodes captured screen frames and streams them live to the admin browser via TURN relay
- `scrap` – cross-platform screen frame grabber used specifically for real-time frame capture at low latency for the live view stream
- `tokio-tungstenite` – async WebSocket client used by the daemon to connect to the backend signaling server for WebRTC offer/answer exchange

Screenshot Compression

- `image` – used to resize screenshots to a maximum resolution (e.g., 1920×1080) and re-encode them as WebP or JPEG before uploading to S3

Local Queue / Offline Storage

- `rusqlite` – embeds a local SQLite database in the daemon to buffer activity events and screenshot metadata when the device is offline, ensuring no data is lost

Upload

- `request` – async HTTP client used to batch-upload activity events to the backend ingestion API and to PUT screenshots directly to S3 using presigned URLs

IPC

- `axum` (local HTTP) or named pipe – local IPC server that receives commands from the Electron shell (start session, pause monitoring, update config, get status)

Logging

- `tracing` – structured logging framework used throughout the daemon for events like upload failures, idle detection, and session changes
 - `tracing-subscriber` – formats and outputs tracing logs to files or stdout for local debugging and support
-

Rust Backend – Axum + MongoDB

Web Framework

- `axum` – main HTTP web framework for all backend API routes (ingestion, reporting, auth, GitHub webhooks, live view signaling)
- `tower` – middleware stack foundation used by Axum for composing request/response processing pipelines
- `tower-http` – provides ready-made HTTP middleware: CORS headers, request compression, timeout enforcement, and request tracing
- `hyper` – low-level async HTTP server that Axum runs on top of for handling raw TCP connections
- `tokio` – async runtime that powers all concurrent backend tasks: API requests, DB queries, background jobs, and WebSocket connections

Database

- `mongodb` – official async MongoDB Rust driver used to read and write all collections: users, events, sessions, screenshots, aggregates, and GitHub events
- `bson` – MongoDB-native data types used in all document models: ObjectId for IDs, DateTime for timestamps, and Document for raw queries

WebRTC Signaling Service

- `tokio-tungstenite` – async WebSocket server used as the signaling channel: forwards WebRTC offer/answer/ICE candidates between the Rust daemon and the admin browser
- `coturn` or Cloudflare TURN – TURN/STUN relay server that enables WebRTC connections to work through firewalls and NAT; coturn is free and self-hosted (Oracle Cloud free tier), Cloudflare TURN offers 1,000 GB/month free

GitHub Integration Service

- `hmac` + `sha2` – used to verify the `X-Hub-Signature-256` header on every incoming GitHub webhook POST to ensure the payload is genuinely from GitHub and not tampered with
- `octocrab` (optional) – Rust GitHub API client for fetching additional data beyond webhooks such as diff stats, contributor graphs, and repo metadata

Authentication

- `jsonwebtoken` – encodes and decodes JWTs issued at login; tokens carry user id, org id, role, and expiry claims used across all protected routes
- `axum-jwt-auth` – Axum middleware extractor that validates JWTs on every incoming request and rejects unauthorized access automatically
- `argon2` – password hashing algorithm used when storing and verifying employee and admin passwords in MongoDB
- `uuid` – generates unique v4 IDs for users, devices, sessions, events, and screenshot records

Serialization & Validation

- `serde` + `serde_json` – serialize/deserialize all HTTP request and response bodies to/from Rust structs
- `validator` – validates incoming request payloads (required fields, email format, min/max lengths) before processing

File / Object Storage

- `aws-sdk-s3` – official AWS Rust SDK used to generate presigned PUT URLs for daemon screenshot uploads and presigned GET URLs for dashboard screenshot access

Background Jobs / Aggregation

- `tokio-cron-scheduler` or `apalis` – schedules periodic background jobs: daily metrics aggregation, GitHub stats rollups, weekly employee summary email generation, and stale session cleanup

Caching

- `redis` – used for rate limiting on API endpoints, caching org-level policy configs, and storing short-lived session tokens

- `moka` – in-process async cache for frequently read hot data (e.g., user role lookups, org config) to avoid repeated MongoDB queries

Email

- `lettre` – Rust email client used to send transactional emails via SMTP or API: weekly productivity summaries to employees, idle alerts, and attendance notifications

Logging & Observability

- `tracing` + `tracing-subscriber` – structured logging in all backend services with request IDs, user context, and error details
- `opentelemetry` + `opentelemetry-otlp` – exports distributed traces and metrics to Grafana, Datadog, or any OpenTelemetry-compatible backend for full observability
- `axum-prometheus` – exposes a `/metrics` endpoint in Prometheus format for Grafana dashboards to scrape backend performance metrics

Config

- `dotenvy` – loads environment variables from `.env` files for local development and staging environments
 - `envy` – deserializes environment variables directly into typed Rust config structs for type-safe configuration management
-

Web App – Admin + Employee Dashboard

Core

- `react` + `react-dom` – component-based UI for all admin and employee dashboard pages
- `typescript` – type-safe frontend code shared across admin and employee views
- `vite` – fast build tool with HMR for the web dashboard
- `react-router-dom` – handles routing between admin (`/admin/*`) and employee self-service (`/me/*`) pages with role-based guards

UI & Styling

- `@mui/material` or `antd` – dashboard-grade components: data tables, date range pickers, modals, drawers, and navigation menus

- `tailwindcss` – utility CSS for custom layouts, spacing, and responsive design across all pages

Charts

- `recharts` or `echarts-for-react` – renders all productivity graphs, active vs idle timelines, GitHub commit trend bars, and PR activity charts on both admin and employee views
- `react-calendar-heatmap` – renders the attendance calendar with colour-coded presence/absence days and optionally a GitHub-style commit activity heatmap on the employee dashboard

State & Data Fetching

- `@tanstack/react-query` – manages all API data fetching with automatic caching, background refetch, pagination, and loading/error states
- `zustand` – global app state for current user session, selected org, active date filters, and role context shared across components

API Communication

- `axios` – HTTPS client for all REST calls to the backend
- Native `WebSocket` – real-time data for live tracking updates and WebRTC signaling channel used by the live view feature

Live View (Admin only)

- Native `RTCPeerConnection` – establishes WebRTC connection with the employee daemon via TURN relay to receive the live screen stream
- `<video>` element – renders the incoming stream as a read-only video; no keyboard or mouse interaction is ever sent back

GitHub Activity Feed

- `@primer/octicons-react` – renders GitHub-appropriate icons for each event type: push arrow, PR merge icon, review checkmark, issue circle, branch icon
- `react-syntax-highlighter` (optional) – displays commit messages or PR description code snippets with syntax highlighting inline in the feed

Screenshot Gallery

- `react-photo-album` or `yet-another-react-lightbox` – renders the screenshot gallery grid and provides a full-screen lightbox with zoom, navigation, and timeline context for admin and employee views

Forms & Validation

- `react-hook-form` – manages form state for login, settings, profile, and org configuration forms with minimal re-renders
- `zod` – schema validation for all form inputs; can be shared with backend TypeScript types for consistent validation rules

Export & Reports

- `jspdf` + `html2canvas` – generates downloadable PDF reports from dashboard views directly in the browser for time logs and productivity summaries
- `exceljs` – generates downloadable CSV/Excel files for attendance records and detailed time logs

Role-Based Access

- `jwt-decode` – decodes the JWT on the client after login to extract user id, role (admin/manager/employee), org id, and token expiry without making an API call
 - `@casl/ability` – defines centralized permission rules per role: what each role can view, create, or manage across all pages and features
 - `@casl/react` – provides the `<Can>` React component and `useAbility` hook to conditionally show or hide UI elements, buttons, and routes based on the current user's permissions
-

DevOps & Infra

- `Docker` – containerizes each Rust backend service into isolated images for consistent builds and deployments across environments
- `docker-compose` – orchestrates the full local dev environment: MongoDB, backend services, MinIO (S3), and coturn all in one command
- `Kubernetes` or `Nomad` – manages production container orchestration: scaling, rolling deployments, health checks, and service discovery
- `GitHub Actions` – CI/CD pipelines that build Rust backend binaries, Electron installers per OS, and Docker images on every push or release

- `MinIO` – self-hosted S3-compatible object storage used in local development and staging to test screenshot upload/download without touching production S3
- `MongoDB Atlas` – managed cloud MongoDB cluster for production; handles replication, backups, and scaling automatically
- `coturn` – open-source self-hosted TURN/STUN relay server deployed on Oracle Cloud free tier or a \$5/month VPS for WebRTC NAT traversal
- `ngrok` – exposes the local backend to the internet during development so GitHub webhooks can reach the local machine for testing
- `Grafana` – visualizes backend metrics and system health dashboards using data from Prometheus and OpenTelemetry
- `Prometheus` – scrapes and stores time-series metrics from all backend services via the `/metrics` endpoint exposed by `axum-prometheus`
- `Sentry` – captures and reports runtime errors and crashes from both the Electron app and backend services with full stack traces
- Email provider (`SendGrid` , `Resend` , or `AWS SES`) – delivers transactional emails sent by the backend: weekly employee summaries, idle alerts, and attendance notifications